

Agent Architecture Design Specification

A lean, case-centered cloud automation agent with durable memory, knowledge retrieval and controlled tool execution.

The agent may reason and act, but the case database remains the memory. Every extracted fact, run, tool call and result must be traceable.

PURPOSE	SCOPE	STATUS
Turn incoming questions and tasks into reliable, stored and repeatable work.	Knowledge answers, case extraction, asynchronous task execution and run history.	Revised execution draft Version 2.0

Prepared for staged implementation and cloud hosting

01 / REVISED DESIGN SPEC

A small agent, not a large support platform

The first release serves a modest number of concurrent users. Its job is to receive work, understand it, use approved knowledge and tools, and preserve a durable case record.

One case is the container for context. Questions, extracted details, tasks, runs, outputs and follow-ups all attach to that case.

1. Accept work

Receive a question or task through a small web screen or API. Create a new case or attach the input to an existing case.

2. Understand intent

Classify the request as a knowledge question, automation task, case update, clarification need or unsupported request.

3. Extract details

Convert unstructured case text into typed fields. Store the raw extraction, normalized values, confidence and source provenance.

4. Execute safely

Build a plan and let a worker call only approved tools. Validate outputs, record each step and request approval when required.

Explicitly out of scope for the first release

No enterprise CRM

No large support department, complex assignment queues, multi-region routing or broad customer-service portal.

No premature infrastructure

No microservices, Kafka, dedicated vector database, Kubernetes or self-hosted model fleet without measured need.

No unrestricted autonomy

The agent cannot invent authoritative facts or perform sensitive, irreversible or external actions outside the tool policy.

No prompt-only memory

Case facts and task state are never trusted only to conversation history or an LLM context window.

02 / REVISED DESIGN SPEC

Seven rules keep the agent simple and dependable

These constraints guide implementation independently of model provider or hosting vendor.

1. Case memory first

The database, not the model, stores what the case is, what was learned and what happened.

2. Extracted data is evidence

Keep the original value, normalized value, type, source, confidence, validation state and extraction run.

3. Retrieval before generation

Knowledge answers use only published document versions and return citations. Missing evidence produces a clear fallback.

4. The harness controls the model

The LLM proposes intent, fields, plans and text. Typed schemas, policy checks and deterministic code accept or reject them.

5. Tools are allowlisted

Every tool has a narrow input schema, timeout, retry rule, permission level and auditable result.

6. Jobs are durable and idempotent

A worker may restart without losing the task or repeating an external side effect. Each action carries an idempotency key.

7. Scale only after measurement

Begin with one web process, one worker and one database. Add capacity when queue age, latency or provider limits require it.

System boundary

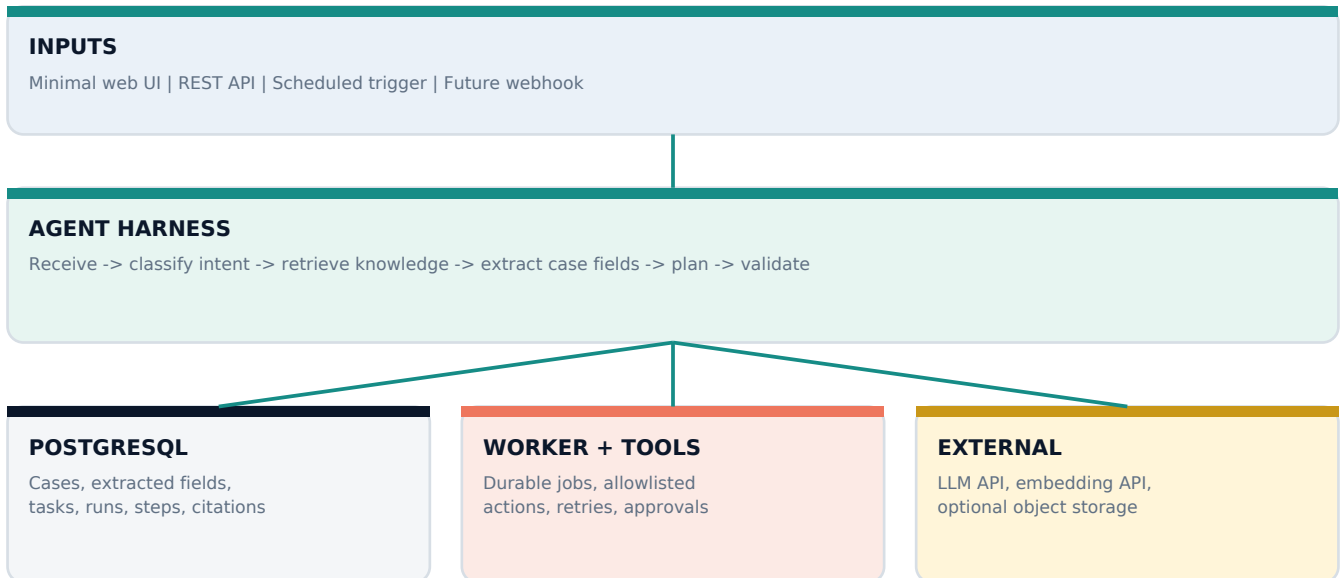
The system may summarize records, retrieve procedures, extract case details, draft solutions and execute configured automations. It must not claim that an external event occurred unless a verified tool result or human confirmation proves it.

DECISION	LLM MAY PROPOSE	SOFTWARE MUST ENFORCE
Intent	A category and confidence	Allowed category, schema and fallback
Case fields	Typed values from the input	Validation, provenance and version history
Plan	Ordered steps and tool choices	Policy, approval, limits and idempotency
Answer	Grounded response text	Citation presence and active-source check
Completion	A summary of work	Verified tool outcomes and stored final state

03 / REVISED DESIGN SPEC

The lean production architecture

A single Python codebase is deployed as a small web process and a separate worker process. Both use the same domain services and database.



One codebase, two process types: web and worker. PostgreSQL is the durable system of record.

Recommended first-release shape

PART	RESPONSIBILITY	FIRST-RELEASE CHOICE
Web/API	Accept requests, return answers, expose case and job status	FastAPI with a minimal responsive UI or API client
Agent harness	Intent, retrieval, extraction, planning, validation and response assembly	Python modules with Pydantic input and output schemas
Worker	Execute queued tasks and slow ingestion outside the request path	One worker process from the same container image
Database	Durable cases, fields, jobs, runs, citations and audit events	Managed PostgreSQL; pgvector only if semantic retrieval is required
Job queue	Lease pending work, retry failures and resume after restart	PostgreSQL-backed job table initially; Redis is optional later
Files	Retain source documents and attachments	Object storage for large files; hashes and metadata remain in PostgreSQL
Models	Generate embeddings and structured or natural-language outputs	Provider adapters around managed LLM and embedding APIs

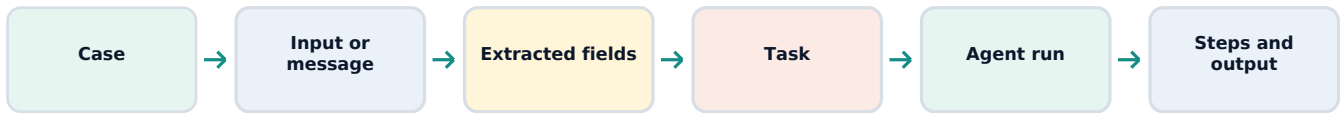
All extracted case information is stored in PostgreSQL. Large original files may live in object storage, but the database retains their case link, filename, content type, size, hash, storage key and extraction history.

04 / REVISED DESIGN SPEC

Durable case memory and execution records

The schema must preserve both the latest usable state and the history needed to explain how that state was produced.

Case-centered relationship



RECORD	MINIMUM CONTENT	WHY IT EXISTS
Case	id, external reference, title, status, source channel, owner, timestamps	Container for all related work and knowledge
CaseInput	case id, raw text, sender, attachment references, received time	Preserves exactly what entered the system
ExtractedField	name, data type, raw value, normalized JSON value, source input, confidence, validation, version	Stores every extracted detail with provenance
Task	case id, task type, instructions, status, approval requirement, idempotency key	Durable unit of automation
AgentRun	intent, model, prompt version, status, timings, tokens, cost estimate, error	One reasoning or generation attempt
RunStep	sequence, action, tool, arguments, result, status, retry count, timestamps	Auditable execution trace
Knowledge records	document, version, chunk, embedding metadata and publication status	Versioned retrieval source
Citation	run or output, document version, chunk, quoted span or locator	Explains the evidence used
Approval	task, requested action, requester, approver, decision, reason and time	Human control point for sensitive work
CaseEvent	actor, event type, previous state, new state, payload and time	Append-only case history

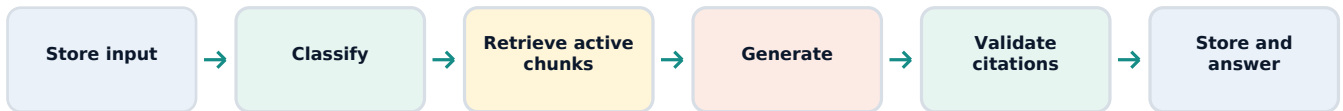
Extraction is versioned, not overwritten. A corrected value creates a new field revision and records who or what accepted the correction.

05 / REVISED DESIGN SPEC

Two primary workflows define the product

Knowledge questions should be fast and grounded. Automation tasks may continue asynchronously and expose durable progress.

Knowledge question



- Create or identify the case before calling the model.
- Retrieve only from active knowledge versions and store the selected chunk identifiers.
- If retrieval is weak or citations fail validation, answer with a bounded fallback instead of guessing.

Automation task



- Validate extracted values against the schema for that task type.
- Set the task to WAITING_FOR_INPUT when a required value is missing or ambiguous.
- Store the plan before execution; each step then stores arguments, result and status.
- External side effects require idempotency keys and may require human approval.

Follow-up and recovery

Follow-up input

Attach new information to the case, run extraction again, propose field revisions and resume any task that was waiting for input.

Worker restart

The next worker leases the pending or expired job from PostgreSQL and continues only from the last safely committed step.

Tool failure

Retry transient failures with limits. Persist terminal errors and return a visible error reference instead of claiming success.

Human correction

Record the correction as a new revision and retain the original model extraction for audit and later evaluation.

06 / REVISED DESIGN SPEC

The harness turns an LLM into a controlled agent

Provider calls are replaceable adapters. The application owns prompts, schemas, memory, permissions, tool access and completion rules.

HARNESS PART	CONTRACT
Input guard	Size limits, accepted file types, authentication, rate limit and basic prompt-injection indicators
Intent router	Returns a small enum plus confidence and reason: KNOWLEDGE_QUERY, AUTOMATION_TASK, CASE_UPDATE, NEEDS_CLARIFICATION or UNSUPPORTED
Case extractor	Returns task-specific structured JSON validated by Pydantic; persists raw response and accepted fields
Retriever	Searches active chunks, applies score and document filters, and returns traceable citations
Planner	Produces bounded steps referencing only registered tools and explicit case fields
Policy gate	Checks role, approval class, destination, amount or sensitivity, step count, timeout and allowed tool
Tool runner	Executes typed adapters with idempotency, timeouts, retries and sanitized result storage
Output validator	Confirms required evidence, expected result schema, case updates and user-safe response

Minimal application interfaces

INTERFACE	PURPOSE
POST /cases	Create a case from text and optional attachments
POST /cases/{id}/messages	Add information or a follow-up question
POST /cases/{id}/tasks	Create and queue an automation task
GET /cases/{id}	Return case state, accepted fields, messages, tasks and outputs
GET /tasks/{id}	Return durable progress, waiting reason, result or error reference
POST /tasks/{id}/approvals	Approve or reject a guarded action
POST /knowledge/documents	Upload a knowledge source for processing
POST /knowledge/versions/{id}/publish	Activate a processed version for retrieval

07 / REVISED DESIGN SPEC

Hosting for modest concurrency

Choose the lowest-operational-burden platform that can run a web container, a worker process and managed PostgreSQL. Keep deployment provider-neutral until budget and region are confirmed.

Web process

One small always-on instance. Handles HTTPS requests, authentication, case APIs, knowledge questions and status polling.

Worker process

One small instance with controlled concurrency, initially 1 to 4 jobs depending on tool and LLM limits.

Managed PostgreSQL

Automated backups, encrypted transport and enough storage for case history, structured results and retrieval metadata.

Optional object storage

Use for original PDFs and large attachments. Do not rely on a container filesystem for durable files.

Operational baseline

AREA	FIRST-RELEASE REQUIREMENT
Health	Web readiness, database connection, worker heartbeat and oldest queued-job age
Reliability	Durable job leasing, bounded retry, dead-letter status, timeouts and graceful worker shutdown
Security	Secrets outside code, least-privilege database and tool credentials, staff authentication, file validation and encrypted transport
Privacy	PII masking in logs, field-level access rules, retention categories and a deliberate deletion workflow
Observability	Request ID, case ID, task ID and run ID in structured logs; LLM latency, tokens, estimated cost and failure category
Recovery	Automated database backups, restore test, versioned deployment image and documented rollback

Scale only when one of these persists

Queue pressure

Oldest pending task exceeds the agreed target or users regularly wait behind unrelated work.

Provider pressure

LLM rate limits, tool limits or response time dominate the task duration.

Compute pressure

Web or worker CPU and memory remain constrained after simple optimization.

Data pressure

Database queries or vector retrieval miss measured latency goals after indexing and query review.

08 / REVISED DESIGN SPEC

Implementation plan and exit checks

Build a thin vertical slice first, then add knowledge and tool execution without changing the case-centered foundation.

STAGE	BUILD	EXIT CHECK
0. Foundation	Repository, typed configuration, container image, FastAPI health route, managed PostgreSQL connection, migrations, request IDs and a hosted staging environment.	Web and worker deploy from one commit; health and worker heartbeat are visible.
1. Case memory	Case, input, extracted field, event, task, run and step tables. Create-case API and one typed extraction schema.	A submitted case survives restart; every extracted field has value, source and extraction run.
2. Knowledge retrieval	Document versions, text extraction, chunks, embeddings, active-version search, cited answer generation and fallback.	A question returns a stored, cited answer or a clear no-evidence response.
3. Task agent	Intent router, planner, durable queue, worker, one real allowlisted tool, idempotency, retry and result validation.	A task runs asynchronously and its exact steps and output can be reconstructed.
4. Minimal interface	Simple operator screen for submitting work, viewing case fields, answering clarification requests, approving guarded steps and reviewing results.	A non-developer can complete the full question and task flows without database access.
5. Launch hardening	Authentication, permissions, masked logs, backups, restore test, error handling, cost controls, realistic end-to-end tests and production deployment.	Recovery is demonstrated and all agreed safety and workflow tests pass.

Definition of done for the first useful release

Durable intake A question or task creates or updates a case and preserves the original input.	Complete extraction All accepted case details are queryable in PostgreSQL with confidence, source and revision history.
Grounded knowledge Answers use active knowledge, store citations and refuse unsupported claims.	Visible automation A worker executes at least one useful tool workflow and exposes pending, waiting, failed and completed states.
Controlled action Sensitive steps pause for approval and external effects are protected from accidental duplication.	Recoverable service The hosted system can restart, restore its database and resume durable work without losing the case history.

Start with Stage 0 and one real task type. The architecture should prove that single workflow end to end before more tools, channels or agents are added.